CASE STUDY 4: MULTI-PRODUCT SHOPPING BUDGET

1. Problem Statement

Multi-Product Shopping Budget is a simple application used to calculate the total cost of multiple products purchased by a customer. The program accepts the product prices and quantities, calculates the total shopping cost, and checks whether the total amount is within the customer's budget.

2. Objectives

- To develop a Python program to calculate shopping expenses.
- To calculate the total cost of multiple products.
- To compare the total cost with the available budget.
- To display whether the shopping is within or above the budget.
- To understand loops, lists, conditional statements, and arithmetic operations in Python.

3. Assumptions

1. The customer purchases three different products.
2. The customer enters the price and quantity of each product.
3. Product prices and quantities must be positive.
4. The customer enters a positive shopping budget.
5. The total cost is calculated by multiplying each product's price by its quantity.
6. The program compares the total cost with the available budget.

4. Algorithm

1. Start the program.
2. Ask the user to enter the shopping budget.
3. Initialize the total cost to zero.
4. Accept the price and quantity of three products.
5. Check whether the budget, prices, and quantities are valid.
6. Calculate the cost of each product using price × quantity.
7. Add all product costs to calculate the total cost.
8. Display the total shopping cost.
9. Compare the total cost with the budget.
10. If the total cost is less than or equal to the budget, display "Shopping is within budget".

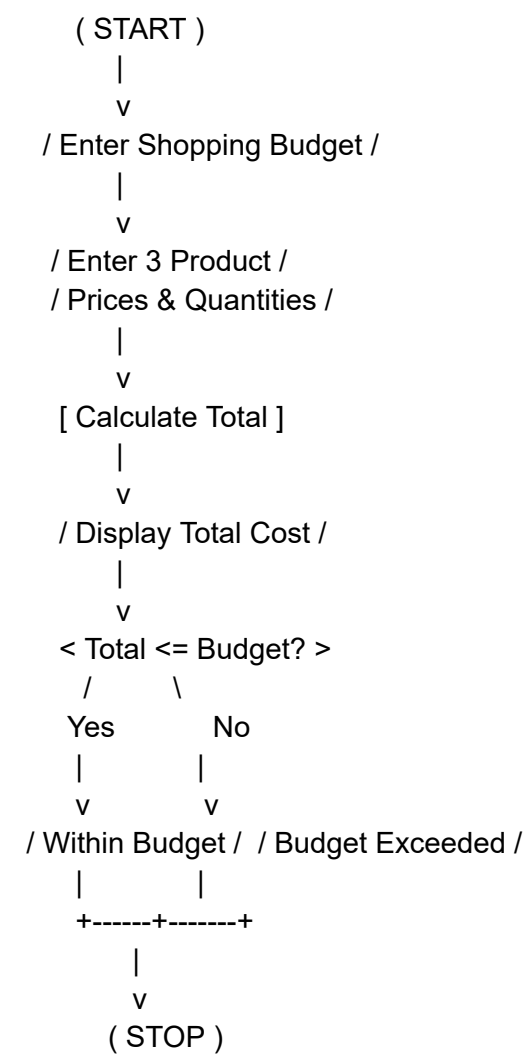
11. Otherwise, display "Budget exceeded".
12. Stop the program.

5. Flowchart

Flowchart symbols:

- Oval – Start/Stop
- Parallelogram – Input/Output
- Rectangle – Process
- Diamond – Decision
- Arrow – Direction of flow

Simple flowchart to draw in your record:



Draw the Start and Stop in oval shapes, inputs and outputs in parallelograms, calculations in rectangles, and the budget comparison in a diamond.

6. Pseudocode

```
START

INPUT budget
total = 0

REPEAT 3 TIMES
   INPUT price
   INPUT quantity

   IF price <= 0 OR quantity <= 0
      DISPLAY "Invalid Input"
      STOP
   END IF

   cost = price * quantity
   total = total + cost
END REPEAT

DISPLAY total

IF total <= budget
   DISPLAY "Shopping is within budget"
ELSE
   DISPLAY "Budget exceeded"
END IF

STOP
```

7. Python Source Code

```
budget = float(input("Enter your budget: "))
total = 0

if budget <= 0:
   print("Invalid Budget")

else:
   valid = True

   for i in range(1, 4):
      price = float(input("Enter product price: "))
      quantity = int(input("Enter product quantity: "))

      if price <= 0 or quantity <= 0:
         print("Invalid Input")
         valid = False
         break
```

```
        total = total + (price * quantity)

    if valid:
        print("Total Shopping Cost:", total)

        if total <= budget:
            print("Shopping is within budget")
        else:
            print("Budget exceeded")
```

8. Test Cases and Expected Outputs

Test Case 1: Shopping Within Budget

Input:

```
Enter your budget: 2000
Enter product price: 100
Enter product quantity: 2
Enter product price: 200
Enter product quantity: 3
Enter product price: 300
Enter product quantity: 1
```

Expected Output:

```
Total Shopping Cost: 1100.0
Shopping is within budget
```

Test Case 2: Budget Exceeded

Input:

```
Enter your budget: 1000
Enter product price: 300
Enter product quantity: 2
Enter product price: 200
Enter product quantity: 2
Enter product price: 150
Enter product quantity: 2
```

Expected Output:

```
Total Shopping Cost: 1300.0
Budget exceeded
```

Test Case 3: Invalid Product Price

Input:

Enter your budget: 2000
Enter product price: -100
Enter product quantity: 2

Expected Output:

Invalid Input

Note: Run each test case separately. Enter all three products for valid shopping test cases.

9. Output Screenshots

```
budget = float(input("Enter your budget: "))
total = 0

if budget <= 0:
    print("Invalid Budget")

else:
    valid = True

    for i in range(1, 4):
        price = float(input("Enter product price: "))
        quantity = int(input("Enter product quantity: "))

        if price <= 0 or quantity <= 0:
            print("Invalid Input")
            valid = False
            break

        total = total + (price * quantity)

    if valid:
        print("Total Shopping Cost:", total)

        if total <= budget:
            print("Shopping is within budget")
        else:
            print("Budget exceeded")
```

```
Python 3.15.0 (tags/v3.15.0:d220409, Oct  9 2026, 11:55:56) [MSC v.1951 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case4.
py
Enter your budget: 2000
Enter product price: 100
Enter product quantity: 2
Enter product price: 200
Enter product quantity: 3
Enter product price: 300
Enter product quantity: 1
Total Shopping Cost: 1100.0
Shopping is within budget
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case4.
py
Enter your budget: 1000
Enter product price: 300
Enter product quantity: 2
Enter product price: 200
Enter product quantity: 2
Enter product price: 150
Enter product quantity: 2
Total Shopping Cost: 1300.0
Budget exceeded
>>
= RESTART: C:/Users/sirit/OneDrive/Desktop/Python-Activity-Based-Learning/case4.
py
Enter your budget: 2000
Enter product price: -100
Enter product quantity: 2
Invalid Input
>>
```

10. Conclusion

Thus, the Multi-Product Shopping Budget program was successfully implemented using Python. The program calculates the total cost of multiple products and compares it with the customer's budget. It also handles invalid budgets, prices, and quantities. This case study demonstrates the use of loops, conditional statements, and arithmetic operations in Python.